Highway Network Notebook

Line-by-Line Explanation

CIFAR-10 convolutional highway network

Depth-20 convention, 19 highway blocks, 2,714,826 parameters

Purpose. This document explains the notebook cell by cell and, for code cells, line by line. It answers two questions for every line: what the line does, and why it is written that way. It is meant as preparation for questions about the implementation, not as a replacement for the paper or the notebook.

Notebook	`highway_cifar10.ipynb`
Model	Convolutional highway network, 19 highway blocks (depth-20 hidden stack)
Hidden blocks	19, split `6/6/7` over three stages
Physical convolutions	41, including stem and downsampling
Parameters	2,714,826
Training	SGD, 25 epochs, batch 128, LR 0.1, decay 0.89
Runtime	About 13 to 15 minutes on an RTX 3060 Ti

Contents

How to read this document . . . 4
Executive summary . . . 4
What the notebook does . . . 4
What it is not . . . 4
Architecture at a glance . . . 4
Cell 0: The highway layer: theory reference . . . 4
Cell 1: Setup heading . . . 5
Cell 2: Imports and device selection . . . 5
Cell 3: Configuration and seeding . . . 5
Cell 4: Data heading . . . 6
Cell 5: Dataset, transforms, and loaders . . . 6
Cell 6: The highway layer in code heading . . . 7
Cell 7: The convolutional highway block . . . 7
Cell 8: The convolutional highway network heading . . . 8
Cell 9: The full convolutional network . . . 8
Cell 10: Training setup heading . . . 9
Cell 11: Accuracy, gate statistics, and training loop . . . 9
Cell 12: Train the highway network heading . . . 11
Cell 13: Create and train the highway model . . . 11
Cell 14: Results heading . . . 11
Cell 15: Loss and accuracy curves . . . 11
Cell 16: Gradient flow heading . . . 12
Cell 17: Measure gradient norm at each block input . . . 12
Cell 18: Gate bias and gate activity heading . . . 13
Cell 19: Gate bias and mean activity per layer . . . 13
Cell 20: Gate evolution heading . . . 14
Cell 21: Gate-evolution heatmap . . . 14
Cell 22: Single-image gate map heading . . . 15
Cell 23: Gate map and histogram for one image . . . 15
Cell 24: Class-specific routing heading . . . 15
Cell 25: Class-routing heatmap . . . 15
Cell 26: Information highway heading . . . 16
Cell 27: Layer-to-layer input/output similarity . . . 16
Cell 28: Lesioning heading . . . 17
Cell 29: Layer-lesion experiment . . . 17
Cell 30: Feature space heading . . . 17
Cell 31: t-SNE of pooled features . . . 17
Cell 32: Predictions heading . . . 18
Cell 33: Prediction grid . . . 19
Reference tables and likely questions . . . 19
Tensor shape flow . . . 19
Gate tensor shape . . . 19
Parameter breakdown . . . 19
Hyperparameter rationale . . . 20
Depth counting . . . 20
Likely questions and short answers . . . 20
Why use convolution instead of a fully connected layer? . . . 20
Why is the kernel 3×3 with padding 1? . . . 20
Why can a highway block not change channels? . . . 20
Why 19 blocks? . . . 20
Why are there 41 convolutions if the depth is 20? . . . 21
Why is the transform gate a sigmoid? . . . 21
Why start the gate bias at −2? . . . 21
Why He initialization? . . . 21
Why reset the gate bias after initialization? . . . 21
Why use SGD with momentum instead of Adam? . . . 21
Why learning rate 0.1? . . . 21
Why decay 0.89 instead of 0.98? . . . 21
Why 25 epochs? . . . 21

Why weight decay 5e-4? ... 21
Why gradient clipping at 10? ... 21
Why is there no softmax before the loss? ... 21
Why global average pooling? ... 21
Why is `shuffle=True` only for training? ... 21
Why `pin_memory` only on CUDA? ... 21
Why `set_to_none=True` in `zero_grad`? ... 21
Why `detach` when storing gates? ... 21
Why `retain_grad` in the gradient experiment? ... 22
Why does the gradient helper repeat the downsampling logic? ... 22
Why average the gate map over channels? ... 22
Why does lesioning use accuracy instead of loss? ... 22
Why is t-SNE qualitative? ... 22
Why is there only one model now? ... 22
Known inconsistencies and caveats ... 22
Paper map ... 22
One-paragraph summary to memorize ... 22

How to read this document

- The document follows the notebook in cell order. Each section starts with a heading that names the cell and its role.
- Markdown cells are explained paragraph by paragraph. Code cells show the full numbered source and then a line-by-line explanation.
- In the line-by-line lists, `*Line 12:*` refers to the line number printed in the code block, not to a line in this document.
- A few multi-line expressions are explained as one logical line, for example `*Lines 9-12:*`. In those cases the range names every physical line in the group.
- The phrase **why this value** marks a rationale for a hyperparameter or a design choice.
- At the end there are reference tables, likely questions, known inconsistencies, and a map back to the paper.

Executive summary

What the notebook does

The notebook implements a convolutional highway network for CIFAR-10 and studies its internal gates. It trains one model, not a plain baseline. The model uses a depth-20 hidden stack: one plain convolutional stem followed by 19 highway blocks. The paper's released 19-layer CIFAR-10 network instead has 14 highway layers plus plain convolutions. Each block computes

```
y = H(x) * T(x) + x * (1 - T(x))
```

where `H` and `T` are 3×3 convolutions with padding 1. The gate `T` is a sigmoid, and its bias starts at −2 so the gates begin mostly closed. A plain strided convolution changes channels and resolution between stages. Global average pooling and a linear layer produce the 10 class logits.

What it is not

- It is not the paper's exact FitNet reproduction. The widths, block counts, and training length are compact choices for an 8 GB GPU.
- It does not include a plain baseline. The notebook can show that the highway model trains and how its gates behave, but it cannot by itself prove that highway networks beat plain networks at depth.
- It is not a paper-scale run. The authors' released config uses about 400 epochs; this uses 25, so accuracy and gate convergence are lower.

Architecture at a glance

Part	Shape	Detail
Stem	`(B, 32, 32, 32)`	Plain 3×3 convolution, 3 to 32 channels, ReLU.
Stage 1	`(B, 32, 32, 32)`	6 highway blocks at 32 channels.
Down 1	`(B, 64, 16, 16)`	Plain 3×3 convolution, stride 2, 32 to 64 channels.
Stage 2	`(B, 64, 16, 16)`	6 highway blocks at 64 channels.
Down 2	`(B, 128, 8, 8)`	Plain 3×3 convolution, stride 2, 64 to 128 channels.
Stage 3	`(B, 128, 8, 8)`	7 highway blocks at 128 channels.
Pool	`(B, 128)`	Global average pooling.
Head	`(B, 10)`	Linear classifier, raw logits.

The single most important sentence. A highway block learns a per-unit, per-input interpolation between a transformed input `H(x)` and the original input `x`. When the gate is closed, the block is the identity and gradients can pass through unchanged.

The single most important caveat. Twenty-five epochs is a short run. Gate plots may be noisier and less converged than in the paper's 400-epoch experiments.

Cell 0: The highway layer: theory reference

Role of this cell. This is the reference page for every formula used later. It is not executed. When someone asks why the code looks the way it does, the answer is usually one of the equations here. **The layer equation.** A plain layer is `y = H(x, W_H)`. A highway layer adds two gates: `y = H(x, W_H) * T(x, W_T) + x * C(x, W_C)`. `T` is the transform gate and `C` is the carry gate. The paper sets `C = 1 - T`, so the layer becomes `y = H(x) * T(x) + x * (1 - T(x))`. **Why two gates.** The gate decides, per unit, how much of the output comes from the nonlinear transform and how much is copied from the input. This is a learned interpolation, not a fixed residual connection. **The sigmoid gate.** The paper defines `T(x) = sigmoid(W_T^T x + b_T)`. The sigmoid keeps every gate value strictly between 0 and 1, so the layer can move smoothly between copying and transforming. **The two limiting cases.** If `T(x) = 0`, then `y = x`, so the layer is the identity. If `T(x) = 1`, then `y = H(x)`, so the layer is a plain layer. The code never reaches 0 or 1 exactly because a sigmoid is asymptotic, but it can get close. **Why the Jacobian matters.** For one unit, `y_i = T_i(x) * H_i(x)`

`+ (1 - T_i(x)) * x_i`. With a completely closed gate, `T_i(x) = 0`, this becomes `y_i = x_i`. The derivative is `dy_i / dx_j = delta_ij`, so the Jacobian is the identity matrix `I`. A gradient arriving at the layer output is multiplied by `I` on the way back, so it reaches the previous layer unchanged. This is the information highway. **The precise Jacobian caveat.** A real sigmoid is never exactly zero and depends on `x`. The exact Jacobian is `J = diag(T) * H'(x) + diag(1 - T) + diag(H(x) - x) * T'(x)`. The identity is the ideal, locally constant-gate case. Even when the last term is nonzero, the `diag(1 - T)` term keeps a direct gradient path. **Shape requirement.** Equation (3) adds `H(x)` and `x`, so `x`, `y`, `H(x)`, and `T(x)` must have the same shape. This is why the network starts with a plain stem and uses plain layers when changing channels or resolution. **Negative gate bias.** The paper starts the gate bias at a negative value such as −1, −2, or −3. The notebook uses −2, so `sigmoid(-2) ≈ 0.12`. The network starts biased toward carrying, behaving like a shallower network, and the gates open during training where transforming helps. **How to use this cell in a presentation.** Point to the equation, then to the two limiting cases, then to the Jacobian argument. The rest of the notebook is evidence about whether the gates actually behave this way.

Cell 1: Setup heading

Role of this cell. A section heading only. It separates the theory from the executable setup.

Cell 2: Imports and device selection

What this cell does. Loads every library used later and chooses the GPU or CPU. It prints the PyTorch version and the GPU name so the run is reproducible and the hardware is known.

```
 1 | import random
 2 | import time
 3 | from pathlib import Path
 4 |
 5 | import matplotlib.pyplot as plt
 6 | import torch
 7 | import torch.nn as nn
 8 | import torch.nn.functional as F
 9 | from sklearn.manifold import TSNE
10 | from torch.utils.data import DataLoader
11 | from torchvision import datasets, transforms
12 |
13 | print("torch", torch.__version__, "| CUDA available:", torch.cuda.is_available())
14 | device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
15 | if device.type == "cuda":
16 |     print(torch.cuda.get_device_name(0))
```

Line by line:

- **Line 1:** `import random` loads Python's random-number generator, used by `set_seed` for reproducibility.
- **Line 2:** `import time` is used to measure how long training takes.
- **Line 3:** `from pathlib import Path` creates the `data` directory path in a platform-independent way.
- **Line 5:** `import matplotlib.pyplot as plt` is the plotting library used for every figure.
- **Line 6:** `import torch` loads PyTorch: tensors, autograd, and device handling.
- **Line 7:** `import torch.nn as nn` loads neural-network building blocks, including `Conv2d`, `Linear`, and `CrossEntropyLoss`.
- **Line 8:** `import torch.nn.functional as F` loads functional operations used directly, such as `relu`, `sigmoid`, `cosine_similarity`, and `cross_entropy`.
- **Line 9:** `from sklearn.manifold import TSNE` loads scikit-learn's t-SNE implementation for the feature-space plot.
- **Line 10:** `from torch.utils.data import DataLoader` loads the batching and shuffling utility.
- **Line 11:** `from torchvision import datasets, transforms` loads CIFAR-10 and the image preprocessing pipeline.
- **Line 13:** Prints the PyTorch version and whether CUDA is available. This confirms the environment before training.
- **Line 14:** Selects `cuda` if PyTorch can see a GPU, otherwise `cpu`. Every model and batch is moved to this device later.
- **Line 15:** Checks whether the selected device is CUDA.
- **Line 16:** If CUDA is selected, prints the GPU name, which confirms the RTX 3060 Ti is being used.

Cell 3: Configuration and seeding

What this cell does. Defines the small set of values that control the run and defines `set_seed`. The configuration is deliberately short: architecture details live with the model, and optimizer details live inside `train`.

```
 1 | SEED = 0
 2 | DATA_ROOT = Path("data")
 3 | BATCH_SIZE = 128
 4 | EPOCHS = 25
 5 | LR = 0.1  # high CIFAR-10 rate; the highway shortcuts keep SGD stable
 6 | GATE_BIAS = -2.0
 7 | NUM_WORKERS = 2  # set to 0 on Windows if the data loader hangs
 8 |
 9 |
10 | def set_seed(seed=SEED):
11 |     random.seed(seed)
```

```
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
```

Line by line:

- **Line 1:** `SEED = 0` fixes the random seed so initialization and data order are repeatable. Zero is a conventional, easy-to-quote choice.
- **Line 2:** `DATA_ROOT = Path("data")` puts the CIFAR-10 download in `./data`, which is git-ignored.
- **Line 3:** `BATCH_SIZE = 128` is the number of images per training step. The paper does not report a batch size; 128 is the value used by the public reproduction and is standard for SGD.
- **Line 4:** `EPOCHS = 25` means 25 full passes over the 50,000 training images. This is a short pedagogical run; the authors' released Caffe config uses about 400 epochs.
- **Line 5:** `LR = 0.1` is a high CIFAR-10 learning rate that stays stable here because the highway shortcuts preserve gradient flow. The authors' released config uses base LR 0.025 with Nesterov momentum and step decay.
- **Line 6:** `GATE_BIAS = -2.0` is the initial transform-gate bias. `sigmoid(-2)` ≈ `0.12`, so gates start mostly closed, as recommended in Section 2.2 of the paper.
- **Line 7:** `NUM_WORKERS = 2` uses two background processes for data loading. Set it to 0 on Windows if the loader hangs.
- **Lines 10-13:** `set_seed` seeds Python, PyTorch's CPU generator, and every CUDA generator. Calling it before model creation makes runs comparable.

Cell 4: Data heading

Role of this cell. Explains the dataset and the augmentation choices. CIFAR-10 has 50,000 training and 10,000 test images of size 32×32 in 10 classes. Random crops and flips approximate the paper's random translations and mirroring. Both splits are normalized with the dataset mean and standard deviation.

Cell 5: Dataset, transforms, and loaders

What this cell does. Builds the CIFAR-10 training and test pipelines, wraps them in `DataLoader` objects, and saves a fixed probe batch for later gate and gradient analysis.

```
MEAN = (0.4914, 0.4822, 0.4465)
STD = (0.2470, 0.2435, 0.2616)

train_tf = transforms.Compose(
    [
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(MEAN, STD),
    ]
)
test_tf = transforms.Compose([transforms.ToTensor(), transforms.Normalize(MEAN, STD)])

train_set = datasets.CIFAR10(DATA_ROOT, train=True, download=True, transform=train_tf)
test_set = datasets.CIFAR10(DATA_ROOT, train=False, download=True, transform=test_tf)
classes = test_set.classes

train_loader = DataLoader(
    train_set,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS,
    pin_memory=device.type == "cuda",
)
test_loader = DataLoader(
    test_set,
    batch_size=512,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=device.type == "cuda",
)

# A fixed batch used later to inspect the gates.
probe_iter = iter(test_loader)  # keep the iterator alive for a clean worker shutdown
probe_x, _ = next(probe_iter)
probe_x = probe_x[:256].to(device)

print(f"{len(train_set):,} train / {len(test_set):,} test images")
```

Line by line:

- **Line 1:** `MEAN = (0.4914, 0.4822, 0.4465)` is the per-channel mean of CIFAR-10. It is used to center the inputs.
- **Line 2:** `STD = (0.2470, 0.2435, 0.2616)` is the per-channel standard deviation. It scales the inputs to roughly unit variance.

- **Lines 4-11:** `train_tf` is the training transform pipeline. The order matters: augmentation first, then tensor conversion, then normalization.
- **Line 6:** `RandomCrop(32, padding=4)` pads the image to 40×40 and takes a random 32×32 crop. This is translation augmentation, matching the paper's use of small translations.
- **Line 7:** `RandomHorizontalFlip()` mirrors the image with probability 0.5, matching the paper's mirroring augmentation.
- **Line 8:** `ToTensor()` converts a PIL image to a float tensor, changes H×W×C to C×H×W, and scales values to the range 0 to 1.
- **Line 9:** `Normalize(MEAN, STD)` applies `(x - mean) / std` per channel.
- **Line 12:** `test_tf` contains only tensor conversion and normalization. Test data must not be augmented, so evaluation is deterministic.
- **Line 14:** Creates the CIFAR-10 training set, downloads it if missing, and applies `train_tf`.
- **Line 15:** Creates the test set with `test_tf`.
- **Line 16:** `classes = test_set.classes` stores the 10 class names for plot labels and prediction titles.
- **Lines 18-24:** The training `DataLoader` uses batch size 128, shuffles every epoch, uses two workers, and pins memory only when CUDA is available.
- **Line 20:** `batch_size=BATCH_SIZE` uses the configured 128.
- **Line 21:** `shuffle=True` reshuffles the training data every epoch, which is important for SGD.
- **Line 22:** `num_workers=NUM_WORKERS` loads batches in parallel.
- **Line 23:** `pin_memory=True` only on CUDA can speed up host-to-GPU copies.
- **Lines 25-31:** The test `DataLoader` uses batch size 512 because evaluation has no gradients and can use larger batches. `shuffle=False` keeps the order stable.
- **Lines 33-36:** `probe_x` is a fixed batch of 256 test images moved to the device. The iterator is kept in `probe_iter` so its worker processes shut down cleanly instead of racing with Python's deallocator. The batch is reused for gate statistics, gradient measurements, and layer-similarity plots so those analyses are comparable across calls.
- **Line 38:** Prints the dataset sizes with thousands separators.

Cell 6: The highway layer in code heading

Role of this cell. Explains that the next code cell is the convolutional version of Equation (3). One 3×3 convolution computes `H`, another computes gate logits, and a sigmoid produces `T`. Padding 1 keeps the spatial size unchanged. The full gate tensor is stored for later inspection.

Cell 7: The convolutional highway block

What this cell does. Defines one highway block. It is the central architectural unit of the whole notebook. Both the transform and the gate are 3×3 convolutions with padding 1, so the block preserves channels and spatial resolution and the carry connection is shape-valid.

```
 1 | class Highway(nn.Module):
 2 |     def __init__(self, channels: int, gate_bias: float = GATE_BIAS):
 3 |         super().__init__()
 4 |         self.H = nn.Conv2d(channels, channels, 3, padding=1)
 5 |         self.gate = nn.Conv2d(channels, channels, 3, padding=1)
 6 |         assert self.gate.bias is not None
 7 |         nn.init.constant_(self.gate.bias, gate_bias)  # start biased towards carry
 8 |         self.last_gate = torch.zeros(1)  # filled on every forward pass
 9 |         self.gate_off = False  # used later to switch this layer off
10 |
11 |     def forward(self, x):
12 |         h = F.relu(self.H(x))
13 |         t = torch.sigmoid(self.gate(x))
14 |         if self.gate_off:
15 |             t = torch.zeros_like(t)
16 |         self.last_gate = t.detach()
17 |         return h * t + x * (1 - t)
```

Line by line:

- **Line 1:** `class Highway(nn.Module)` defines a PyTorch module, so its parameters are tracked and moved with the rest of the network.
- **Line 2:** `__init__(self, channels, gate_bias=GATE_BIAS)` takes the number of input and output channels. A highway block cannot change the channel count because it adds the input to the transformed output.
- **Line 3:** `super().__init__()` initializes the base `nn.Module`.
- **Line 4:** `self.H = nn.Conv2d(channels, channels, 3, padding=1)` is the transform branch. It is a 3×3 convolution with padding 1, so the output has the same height, width, and channels as the input.
- **Line 5:** `self.gate = nn.Conv2d(channels, channels, 3, padding=1)` is the gate branch. It produces one logit per channel and spatial position.
- **Line 6:** `assert self.gate.bias is not None` documents and narrows the type. `Conv2d` has a bias by default, and the next line needs it.
- **Line 7:** `nn.init.constant_(self.gate.bias, gate_bias)` sets every gate bias to −2. This is the paper's negative initialization, which starts the network biased toward carrying.

- **Line 8:** `self.last_gate = torch.zeros(1)` is a placeholder for the most recent gate tensor. It is overwritten on every forward pass and is used by the analysis cells.
- **Line 9:** `self.gate_off = False` is a switch used by the lesioning experiment. When true, the block is forced to copy its input.
- **Line 11:** `def forward(self, x)` defines the forward pass.
- **Line 12:** `h = F.relu(self.H(x))` computes the transformed state `H(x)` and applies ReLU. ReLU is the paper's CIFAR-10 activation.
- **Line 13:** `t = torch.sigmoid(self.gate(x))` turns the gate logits into values between 0 and 1.
- **Lines 14-15:** If `gate_off` is true, `t` is replaced by zeros. This is the lesioning mechanism: a closed gate forces `y = x`.
- **Line 16:** `self.last_gate = t.detach()` stores the gate tensor without autograd history, so the analysis cells can inspect it without keeping a computation graph alive.
- **Line 17:** `return h * t + x * (1 - t)` implements `y = H(x) * T(x) + x * (1 - T(x))` elementwise. `h * t` is the transformed part and `x * (1 - t)` is the carried part.

Cell 8: The convolutional highway network heading

Role of this cell. Describes the full model: a plain 3×3 stem, two plain strided convolutions that halve the spatial size and change channels, nineteen highway blocks split 6/6/7, global average pooling, and a linear classifier. **Depth convention.** Counting the stem, this is a depth-20 hidden stack: one plain layer followed by 19 highway layers. The paper's released 19-layer CIFAR-10 network is different: it interleaves 14 highway layers with plain convolutions. The stage widths are 32, 64, and 128. **Why the transitions are plain.** `H` and `T` both preserve channels and spatial size. Layers that change channels or resolution cannot be highway blocks because the carry addition would have mismatched shapes. Section 2.1 of the paper uses plain layers for this. **Honesty note.** This is a compact variant of the paper's convolutional setup, not an exact FitNet reproduction. **Initialization.** All weights use He initialization. Gate biases are reset to −2 after initialization so the gates start mostly closed.

Cell 9: The full convolutional network

What this cell does. Defines the complete highway network, the parameter counter, and the initialization routine. The model has 19 highway blocks, 41 physical convolution layers (1 stem + 38 block convolutions + 2 downsampling convolutions), one global average pooling step, and one linear classifier.

```
 1 | class Net(nn.Module):
 2 |     def __init__(
 3 |         self,
 4 |         channels: tuple[int, int, int] = (32, 64, 128),
 5 |         blocks_per_stage: tuple[int, int, int] = (6, 6, 7),
 6 |     ):
 7 |         super().__init__()
 8 |         c1, c2, c3 = channels
 9 |         self.stem = nn.Sequential(
10 |             nn.Conv2d(3, c1, 3, padding=1),
11 |             nn.ReLU(),
12 |         )
13 |         self.blocks = nn.ModuleList(
14 |             Highway(ch) for ch, n in zip(channels, blocks_per_stage, strict=True) for _ in range(n)
15 |         )
16 |         self.break1 = blocks_per_stage[0]
17 |         self.break2 = blocks_per_stage[0] + blocks_per_stage[1]
18 |         self.down1 = nn.Sequential(
19 |             nn.Conv2d(c1, c2, 3, stride=2, padding=1),
20 |             nn.ReLU(),
21 |         )
22 |         self.down2 = nn.Sequential(
23 |             nn.Conv2d(c2, c3, 3, stride=2, padding=1),
24 |             nn.ReLU(),
25 |         )
26 |         self.head = nn.Linear(c3, 10)
27 |
28 |     def forward(self, x):
29 |         x = self.stem(x)
30 |         for i, block in enumerate(self.blocks):
31 |             if i == self.break1:
32 |                 x = self.down1(x)
33 |             if i == self.break2:
34 |                 x = self.down2(x)
35 |             x = block(x)
36 |         x = x.mean(dim=(2, 3))  # global average pooling
37 |         return self.head(x)
38 |
39 |
40 | def n_params(model):
41 |     return sum(p.numel() for p in model.parameters())
42 |
43 |
44 | def init_weights(model):
```

```
45 |     """He initialization for every conv and linear layer, then reset the gate biases."""
46 |     for module in model.modules():
47 |         if isinstance(module, (nn.Linear, nn.Conv2d)):
48 |             nn.init.kaiming_normal_(module.weight, nonlinearity="relu")
49 |             if module.bias is not None:
50 |                 nn.init.zeros_(module.bias)
51 |     for module in model.modules():
52 |         if isinstance(module, Highway):
53 |             assert module.gate.bias is not None
54 |             nn.init.constant_(module.gate.bias, GATE_BIAS)
```

Line by line:

- **Line 1:** `class Net(nn.Module)` defines the full model.
- **Lines 2-6:** The constructor takes `channels=(32, 64, 128)` and `blocks_per_stage=(6, 6, 7)`. The block counts sum to 19, forming a depth-20 hidden stack when the stem is included. The paper's 19-layer CIFAR-10 net instead has 14 highway layers plus plain convolutions.
- **Line 7:** `super().__init__()` initializes the base module.
- **Line 8:** `c1, c2, c3 = channels` unpacks the three stage widths.
- **Lines 9-12:** `self.stem` is a plain 3×3 convolution from 3 RGB channels to 32 channels, followed by ReLU. It changes the input dimensionality before any highway block.
- **Lines 13-15:** `self.blocks` builds the 19 highway blocks in stage order. The nested comprehension pairs each channel width with its block count, and `strict=True` checks that the two tuples have the same length.
- **Line 16:** `self.break1 = 6` is the index where stage 1 ends. The first downsampling happens before block index 6.
- **Line 17:** `self.break2 = 12` is the index where stage 2 ends. It is `6 + 6`, so the second downsampling happens before block index 12.
- **Lines 18-21:** `self.down1` is a plain 3×3 convolution with stride 2 from 32 to 64 channels, followed by ReLU. It halves 32×32 to 16×16.
- **Lines 22-25:** `self.down2` is a plain 3×3 convolution with stride 2 from 64 to 128 channels, followed by ReLU. It halves 16×16 to 8×8.
- **Line 26:** `self.head = nn.Linear(c3, 10)` maps the 128 pooled features to 10 class logits.
- **Line 28:** `def forward(self, x)` defines the forward pass.
- **Line 29:** `x = self.stem(x)` applies the stem.
- **Line 30:** The loop visits all 19 highway blocks in order.
- **Lines 31-32:** Before block 6, apply `down1`. This moves from the 32-channel 32×32 stage to the 64-channel 16×16 stage.
- **Lines 33-34:** Before block 12, apply `down2`. This moves from the 64-channel 16×16 stage to the 128-channel 8×8 stage.
- **Line 35:** `x = block(x)` applies the current highway block.
- **Line 36:** `x = x.mean(dim=(2, 3))` is global average pooling. It turns the `(batch, 128, 8, 8)` feature map into `(batch, 128)`.
- **Line 37:** `return self.head(x)` returns 10 logits per image. No softmax is applied because `CrossEntropyLoss` expects logits.
- **Lines 40-41:** `n_params` sums the number of elements in every learnable parameter. The model has 2,714,826 parameters.
- **Lines 44-54:** `init_weights` applies He initialization to every convolution and linear layer, zeros their biases, and then restores the −2 gate bias for every highway block.
- **Line 48:** `nn.init.kaiming_normal_(module.weight, nonlinearity="relu")` is PyTorch's He normal initialization, appropriate for ReLU networks.
- **Line 49-50:** Biases are zeroed only if they exist. This keeps the function safe for layers without a bias.
- **Lines 51-54:** The second loop finds every `Highway` block and resets its gate bias to −2. This is necessary because the first loop zeros all biases.

Cell 10: Training setup heading

Role of this cell. Lists the paper-inspired training recipe: SGD with momentum 0.9, batch size 128, He initialization, weight decay 5e-4, exponential learning-rate decay by 0.89 per epoch, and 25 epochs. **Why 25 epochs.** The authors' released Caffe config uses 200k iterations, about 400 epochs at batch 100. Twenty-five epochs keeps this pedagogical run near 13 to 15 minutes while still giving the gates time to move. **Why decay 0.89.** The paper's released config uses step decay at 100k/150k/175k iterations, not a per-epoch exponential. For this short run, 0.89 brings the learning rate down to about 0.005. **Why learning rate 0.1.** It is a high CIFAR-10 rate that stays stable here because the highway shortcuts preserve gradient flow. The authors' released config uses base LR 0.025 with Nesterov momentum. **Why weight decay and clipping are mentioned as additions.** The paper does not explicitly state weight decay or gradient clipping. Weight decay 5e-4 is a practical CIFAR-10 choice, and clipping at 10 keeps SGD stable. **Gate tracking.** The mean gate value of every block is recorded after every epoch for the gate-evolution heatmap.

Cell 11: Accuracy, gate statistics, and training loop

What this cell does. Defines three functions: `accuracy` for evaluation, `gate_means_on_batch` for gate statistics, and `train` for the full SGD loop. The training loop is the only place where parameters are updated.

```
1 | @torch.no_grad()
2 | def accuracy(model, loader):
3 |     model.eval()
4 |     correct = total = 0
5 |     for x, y in loader:
6 |         x, y = x.to(device), y.to(device)
```

```
        correct += (model(x).argmax(dim=1) == y).sum().item()
        total += y.numel()
    return correct / total


@torch.no_grad()
def gate_means_on_batch(model, x):
    model.eval()
    model(x)
    return [block.last_gate.mean().item() for block in model.blocks]


def train(model, track_gates: bool = False, epochs: int = EPOCHS, lr: float = LR):
    model.to(device)
    opt = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9, weight_decay=5e-4)
    sched = torch.optim.lr_scheduler.ExponentialLR(opt, gamma=0.89)
    loss_fn = nn.CrossEntropyLoss()

    history = {"loss": [], "acc": [], "gates": []}
    start = time.time()
    for epoch in range(1, epochs + 1):
        model.train()
        running = 0.0
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            opt.zero_grad(set_to_none=True)
            loss = loss_fn(model(x), y)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 10.0)
            opt.step()
            running += loss.item() * y.numel()
        history["loss"].append(running / len(train_set))
        history["acc"].append(accuracy(model, test_loader))
        if track_gates:
            history["gates"].append(gate_means_on_batch(model, probe_x))
        print(
            f"epoch {epoch:3d}/{epochs} | lr {opt.param_groups[0]['lr']:.4f} "
            f"| loss {history['loss'][-1]:.3f} | test acc {history['acc'][-1]:.1%}"
        )
        sched.step()

    print(f"trained in {time.time() - start:.1f}s")
    return history
```

Line by line:

- **Line 1:** `@torch.no_grad()` disables gradient tracking for `accuracy`, saving memory and time during evaluation.
- **Line 2:** `def accuracy(model, loader)` defines the evaluation function.
- **Line 3:** `model.eval()` switches to evaluation mode. This model has no dropout or batch normalization, but `eval()` is still correct practice.
- **Line 4:** `correct = total = 0` initializes the counters for correct predictions and examples.
- **Line 5:** Loops over the loader one batch at a time.
- **Line 6:** Moves the images and labels to the selected device.
- **Line 7:** `model(x).argmax(dim=1)` picks the class with the highest logit. Comparing with `y`, summing, and calling `.item()` counts correct predictions as a Python integer.
- **Line 8:** `total += y.numel()` adds the batch size.
- **Line 9:** Returns the fraction correct.
- **Line 12:** `@torch.no_grad()` disables gradients for gate inspection.
- **Line 13:** `def gate_means_on_batch(model, x)` defines the gate-statistics helper.
- **Line 14:** Switches to evaluation mode.
- **Line 15:** Runs one forward pass so every block stores its `last_gate` tensor.
- **Line 16:** Returns the mean gate value of each block. `mean()` averages over batch, channels, height, and width, producing one number per layer.
- **Line 19:** `def train(model, track_gates=False, epochs=EPOCHS, lr=LR)` defines the training function. `track_gates` controls whether gate means are recorded.
- **Line 20:** Moves the model to the device.
- **Line 21:** Creates SGD with momentum 0.9 and weight decay 5e-4. SGD with momentum is the paper's optimizer.
- **Line 22:** `ExponentialLR(gamma=0.89)` multiplies the learning rate by 0.89 after every epoch. The final rate is about `0.1 * 0.89^25` ≈ `0.0054`.
- **Line 23:** `CrossEntropyLoss` combines log-softmax and negative log-likelihood, so the model outputs raw logits.
- **Line 25:** `history` stores training loss, test accuracy, and optional gate means.
- **Line 26:** Records the start time for the runtime report.
- **Line 27:** Loops over epochs from 1 to `epochs`.
- **Line 28:** `model.train()` switches to training mode.

- **Line 29:** `running = 0.0` accumulates the total loss for the epoch.
- **Line 30:** Loops over training batches.
- **Line 31:** Moves the batch to the device.
- **Line 32:** `opt.zero_grad(set_to_none=True)` clears old gradients. Setting them to `None` is slightly cheaper than zeroing them.
- **Line 33:** Forward pass and cross-entropy loss.
- **Line 34:** Backpropagation computes all gradients.
- **Line 35:** `clip_grad_norm_(..., 10.0)` caps the total gradient norm at 10. This is not in the paper but prevents occasional SGD explosions.
- **Line 36:** `opt.step()` updates the parameters.
- **Line 37:** `running += loss.item() * y.numel()` converts the batch mean loss back to a sum. Dividing by the dataset size later gives the true epoch mean.
- **Line 38:** Appends the epoch's average training loss.
- **Line 39:** Evaluates test accuracy after the epoch.
- **Lines 40-41:** If `track_gates` is true, records the mean gate value of every block on the fixed probe batch.
- **Lines 42-45:** Prints epoch, current learning rate, loss, and test accuracy. `opt.param_groups[0]["lr"]` is the rate actually used during that epoch.
- **Line 46:** `sched.step()` decays the learning rate after the epoch.
- **Line 48:** Prints total training time.
- **Line 49:** Returns the history dictionary.

Cell 12: Train the highway network heading

Role of this cell. A section heading before the model is created and trained.

Cell 13: Create and train the highway model

What this cell does. Seeds the run, creates the 19-block convolutional highway network, applies He initialization and the gate-bias reset, prints the parameter count, and trains for 25 epochs with gate tracking.

```
1 | set_seed()
2 | highway = Net()
3 | init_weights(highway)
4 | print(f"{n_params(highway):,} parameters")
5 | highway_hist = train(highway, track_gates=True)
```

Line by line:

- **Line 1:** `set_seed()` fixes the random seed before model creation.
- **Line 2:** `highway = Net()` creates the model with the default `(32, 64, 128)` channels and `(6, 6, 7)` blocks.
- **Line 3:** `init_weights(highway)` applies He initialization and resets every gate bias to −2.
- **Line 4:** Prints the parameter count. The model has 2,714,826 parameters.
- **Line 5:** Trains for 25 epochs and records gate means after every epoch.

Cell 14: Results heading

Role of this cell. A heading for the training curves. The body now says that there is no plain baseline, so this is not the paper's depth comparison.

Cell 15: Loss and accuracy curves

What this cell does. Plots the highway model's training loss and test accuracy over the 25 epochs.

```
 1 | fig, (ax_loss, ax_acc) = plt.subplots(1, 2, figsize=(11, 3.5))
 2 |
 3 | ax_loss.plot(highway_hist["loss"], color="tab:orange")
 4 | ax_acc.plot([100 * a for a in highway_hist["acc"]], color="tab:orange")
 5 |
 6 | ax_loss.set(xlabel="epoch", ylabel="train loss", title="Training loss")
 7 | ax_acc.set(xlabel="epoch", ylabel="test accuracy (%)", title="Test accuracy")
 8 | for ax in (ax_loss, ax_acc):
 9 |     ax.grid(alpha=0.3)
10 |
11 | plt.tight_layout()
12 | plt.show()
13 |
14 | print(f"final test accuracy: {highway_hist['acc'][-1]:.1%}")
```

Line by line:

- **Line 1:** Creates one figure with two side-by-side axes.
- **Line 3:** Plots training loss against epoch in orange.
- **Line 4:** Plots test accuracy as a percentage. `100 * a` converts fractions to percent.
- **Line 6:** Labels the loss axes: x is epoch, y is training loss.
- **Line 7:** Labels the accuracy axes: x is epoch, y is test accuracy in percent.
- **Lines 8-9:** Adds a light grid to both panels.

- **Line 11:** `tight_layout` prevents label overlap.
- **Line 12:** Displays the figure.
- **Line 14:** Prints the final test accuracy. Because the run is only 25 epochs, this may be below the best value seen in a longer run.

Cell 16: Gradient flow heading

Role of this cell. Explains the gradient-flow experiment. Backpropagation multiplies one Jacobian per layer. Without a shortcut, those products can shrink the gradient with depth. A highway layer's identity path lets gradients pass through more easily. **What the next plot shows.** The gradient norm arriving at each block input, measured once at initialization and once after training.

Cell 17: Measure gradient norm at each block input

What this cell does. Builds a fresh untrained model, runs one batch through the model manually, keeps the gradient of every block input, and plots the norms at initialization versus after training.

```
 1 | def layer_input_grad_norms(model, x, y):
 2 |     model.train()
 3 |     model.zero_grad(set_to_none=True)
 4 |
 5 |     h = model.stem(x)
 6 |     hs = []
 7 |     for i, block in enumerate(model.blocks):
 8 |         if i == model.break1:
 9 |             h = model.down1(h)
10 |         if i == model.break2:
11 |             h = model.down2(h)
12 |         h.retain_grad()
13 |         hs.append(h)
14 |         h = block(h)
15 |
16 |     loss = F.cross_entropy(model.head(h.mean(dim=(2, 3))), y)
17 |     loss.backward()
18 |
19 |     norms = [max(t.grad.norm().item(), 1e-12) for t in hs]
20 |     model.zero_grad(set_to_none=True)
21 |     return norms
22 |
23 |
24 | set_seed()
25 | fresh_highway = Net().to(device)
26 | init_weights(fresh_highway)  # match the initialization used for the trained model
27 |
28 | x_grad, y_grad = next(iter(train_loader))
29 | x_grad, y_grad = x_grad[:256].to(device), y_grad[:256].to(device)
30 |
31 | fig, ax = plt.subplots(figsize=(8, 3.5))
32 | layers = range(1, len(highway.blocks) + 1)
33 | ax.semilogy(
34 |     layers,
35 |     layer_input_grad_norms(fresh_highway, x_grad, y_grad),
36 |     "o-",
37 |     label="at initialization",
38 | )
39 | ax.semilogy(
40 |     layers,
41 |     layer_input_grad_norms(highway, x_grad, y_grad),
42 |     "o-",
43 |     label="after training",
44 | )
45 | highway.eval()
46 | ax.set(
47 |     xlabel="layer",
48 |     ylabel="gradient norm at block input (log scale)",
49 |     title="Gradient reaching each layer",
50 | )
51 | ax.grid(alpha=0.3, which="both")
52 | ax.legend()
53 | plt.tight_layout()
54 | plt.show()
```

Line by line:

- **Line 1:** `def layer_input_grad_norms(model, x, y)` defines the measurement function.
- **Line 2:** `model.train()` puts the model in training mode. There is no dropout or batch normalization, so this mainly matches training conditions.
- **Line 3:** Clears old gradients.

- **Line 5:** Runs the stem to get the input to the first block.
- **Line 6:** `hs = []` will store the tensor that enters each block.
- **Line 7:** Loops over all 19 blocks by index.
- **Lines 8-9:** Before block 6, apply `down1`, matching the model's forward pass.
- **Lines 10-11:** Before block 12, apply `down2`.
- **Line 12:** `h.retain_grad()` keeps the gradient of this non-leaf tensor so it can be read after backpropagation.
- **Line 13:** Appends the block input to `hs`.
- **Line 14:** Applies the block.
- **Line 16:** Applies global average pooling, the classifier, and cross-entropy. Pooling must be included because `Net.forward` does it before the head.
- **Line 17:** Backpropagates through the whole network.
- **Line 19:** For each stored block input, reads the gradient norm. `max(..., 1e-12)` avoids zero values on a log scale.
- **Line 20:** Clears gradients after measuring.
- **Line 21:** Returns one norm per block.
- **Line 24:** Seeds the fresh model.
- **Line 25:** `fresh_highway = Net().to(device)` creates an untrained copy for the initialization measurement.
- **Line 26:** `init_weights(fresh_highway)` applies the same He initialization and gate-bias reset as the trained model. Without it, the "at initialization" curve would use PyTorch's default initialization and would not be comparable.
- **Lines 28-29:** Takes one training batch of 256 images for the gradient estimate.
- **Line 31:** Creates one figure and one axis.
- **Line 32:** `layers` is the 1-based layer index for the x-axis.
- **Lines 33-38:** Plots gradient norms at initialization with circle markers and a log y-axis.
- **Lines 39-44:** Plots gradient norms after training on the same axes.
- **Line 45:** `highway.eval()` restores evaluation mode after the measurement.
- **Lines 46-50:** Sets axis labels and title.
- **Line 51:** Adds a grid for both major and minor ticks.
- **Line 52:** Adds the legend.
- **Lines 53-54:** Lays out and displays the figure.

Cell 18: Gate bias and gate activity heading

Role of this cell. Explains the paper's Figure 2 idea for its 50-layer networks: inspect the learned transform-gate biases and the mean gate activity per layer. A negative bias does not necessarily mean a dead layer. It often makes gates selective: some units stay closed while others open for particular inputs. **What the bars show.** The average gate value of each layer over the whole test set. In this run the biases move toward zero and the gates open on average, which differs from the paper's deeper networks.

Cell 19: Gate bias and mean activity per layer

What this cell does. Computes the mean learned gate bias per layer and the mean gate activity per layer over the test set, then plots both.

```
 1 | def gate_biases(model):
 2 |     return [block.gate.bias.mean().item() for block in model.blocks]
 3 |
 4 |
 5 | @torch.no_grad()
 6 | def gate_activity(model, loader):
 7 |     model.eval()
 8 |     sums = torch.zeros(len(classes), len(model.blocks))
 9 |     counts = torch.zeros(len(classes))
10 |     for x, y in loader:
11 |         model(x.to(device))
12 |         batch_means = torch.stack(
13 |             [block.last_gate.mean(dim=(1, 2, 3)).cpu() for block in model.blocks],
14 |             dim=1,
15 |         )
16 |         for c in range(len(classes)):
17 |             mask = y == c
18 |             sums[c] += batch_means[mask].sum(dim=0)
19 |             counts[c] += mask.sum()
20 |     return sums.sum(dim=0) / counts.sum(), sums / counts[:, None]
21 |
22 |
23 | overall_activity, class_activity = gate_activity(highway, test_loader)
24 | biases = gate_biases(highway)
25 |
26 | fig, (ax_bias, ax_act) = plt.subplots(1, 2, figsize=(11, 3.5))
27 | ax_bias.plot(range(1, len(highway.blocks) + 1), biases, "o-", label="learned")
28 | ax_bias.axhline(GATE_BIAS, color="gray", linestyle="--", label="initial")
29 | ax_bias.set(xlabel="layer", ylabel="gate bias", title="Transform-gate bias per layer")
30 | ax_bias.legend()
31 | ax_bias.grid(alpha=0.3)
32 |
33 | ax_act.bar(range(1, len(highway.blocks) + 1), overall_activity, color="tab:orange")
```

```
ax_act.set(
    xlabel="layer",
    ylabel="mean T",
    title="Mean gate activity on the test set",
    ylim=(0, 1),
)
ax_act.grid(alpha=0.3, axis="y")

plt.tight_layout()
plt.show()
```

Line by line:

- **Line 1:** `def gate_biases(model)` defines the bias helper.
- **Line 2:** For each block, averages the gate-bias vector into one number and returns one value per layer.
- **Line 5:** `@torch.no_grad()` disables gradients for gate statistics.
- **Line 6:** `def gate_activity(model, loader)` defines the activity helper.
- **Line 7:** Switches to evaluation mode.
- **Line 8:** `sums` has shape `(10, 19)`: one total per class and layer.
- **Line 9:** `counts` stores how many examples of each class were seen.
- **Line 10:** Loops over the test loader.
- **Line 11:** Runs the batch so every block stores `last_gate`.
- **Lines 12-15:** For each block, averages gate values over channels and spatial positions, giving one value per sample. Stacks into `(batch, layers)`.
- **Line 16:** Loops over the 10 classes.
- **Line 17:** `mask = y == c` selects the samples of class `c`.
- **Line 18:** Adds those samples' gate activity to the class totals.
- **Line 19:** Adds the sample count.
- **Line 20:** Returns the overall mean per layer and the per-class means with shape `(10, 19)`. `counts[:, None]` makes counts a column for broadcasting.
- **Line 23:** Computes both statistics for the trained highway model.
- **Line 24:** Gets the learned gate biases.
- **Line 26:** Creates two side-by-side panels.
- **Line 27:** Plots learned bias per layer.
- **Line 28:** Draws the initial −2 bias as a dashed reference.
- **Line 29:** Labels the bias axes.
- **Line 30:** Adds the legend.
- **Line 31:** Adds a grid.
- **Line 33:** Bar chart of mean gate activity per layer.
- **Lines 34-39:** Labels the axes, titles the panel, and fixes the y-range to 0 to 1.
- **Line 40:** Adds a horizontal grid at the y-axis ticks.
- **Lines 42-43:** Lays out and displays the figure.

Cell 20: Gate evolution heading

Role of this cell. Explains the gate-evolution heatmap: rows are highway layers, columns are epochs. Bright cells mean the layer transforms its input, dark cells mean it mostly carries. At initialization the gates are near `sigmoid(-2) ≈ 0.12`; the first recorded column is already after one epoch, and in this compact run the gates open on average.

Cell 21: Gate-evolution heatmap

What this cell does. Turns the recorded per-epoch gate means into a layer-by-epoch heatmap.

```
gate_history = torch.tensor(highway_hist["gates"]).T  # layers x epochs

plt.figure(figsize=(8, 4))
plt.imshow(gate_history, aspect="auto", origin="lower", cmap="viridis", vmin=0, vmax=1)
plt.colorbar(label="mean transform gate T")
plt.xlabel("epoch")
plt.ylabel("highway layer")
plt.title("Mean gate value of every layer during training")
plt.tight_layout()
plt.show()
```

Line by line:

- **Line 1:** `highway_hist["gates"]` has shape `(epochs, layers)`. Transposing gives `(layers, epochs)`, which displays naturally as rows equal layers and columns equal epochs.
- **Line 3:** Creates the figure.
- **Line 4:** `imshow` displays the matrix. `aspect="auto"` stretches cells to fit, `origin="lower"` puts layer 1 at the bottom, and `vmin=0, vmax=1` fixes the color scale to the valid gate range.
- **Line 5:** Adds a color bar labeled with the mean transform gate.
- **Line 6:** Labels the x-axis as epoch.
- **Line 7:** Labels the y-axis as highway layer.

- **Line 8:** Adds the title.
- **Lines 9-10:** Lays out and displays the figure.

Cell 22: Single-image gate map heading

Role of this cell. Explains the single-image gate inspection. In a convolutional highway layer, the gate is a tensor with one value per channel and spatial position. The left panel shows one layer's gate map averaged over channels; the right panel shows the distribution of all gate values in that layer. **Why average over channels.** A 64-channel 16×16 gate tensor is too much to display directly. Averaging over channels gives a readable spatial map. The histogram preserves the full distribution.

Cell 23: Gate map and histogram for one image

What this cell does. Runs one test image through the trained model, selects the last block of stage two, and plots that block's gate map and gate histogram.

```
image, label = test_set[0]
with torch.no_grad():
    highway(image.unsqueeze(0).to(device))

layer = highway.break2 - 1  # last block of stage two, 16x16 gates
block = highway.blocks[layer]
assert isinstance(block, Highway)
gate_map = block.last_gate[0].mean(dim=0).cpu()  # average over channels
gates = block.last_gate.cpu()

fig, (ax_map, ax_hist) = plt.subplots(1, 2, figsize=(12, 4), gridspec_kw={"width_ratios": [3, 1]})
im = ax_map.imshow(gate_map, cmap="viridis", vmin=0, vmax=1)
ax_map.set(
    xlabel="width",
    ylabel="height",
    title=f"Layer {layer + 1} gate map for one image ({classes[label]})",
)
fig.colorbar(im, ax=ax_map, label="mean T over channels")

ax_hist.hist(gates.flatten(), bins=50, range=(0, 1), color="tab:orange")
ax_hist.set(xlabel="T", ylabel="values", title="Gate distribution")
plt.tight_layout()
plt.show()
```

Line by line:

- **Line 1:** `image, label = test_set[0]` gets the first test image, already normalized, and its integer label.
- **Lines 2-3:** Runs the image through the model with gradients disabled so every block stores `last_gate`.
- **Line 5:** `layer = highway.break2 - 1` selects the last block of stage two. `break2` is 12, so this is block index 11, the 12th block. It has 16×16 spatial gates.
- **Line 6:** Gets that block.
- **Line 7:** `assert isinstance(block, Highway)` documents and narrows the type.
- **Line 8:** `gate_map = block.last_gate[0].mean(dim=0)` selects the single image and averages over channels, giving a 16×16 map.
- **Line 9:** `gates = block.last_gate.cpu()` keeps the full gate tensor for the histogram.
- **Line 11:** Creates two panels, with the map three times wider than the histogram.
- **Line 12:** Displays the gate map with a fixed 0 to 1 scale.
- **Lines 13-17:** Labels the axes and titles the panel with the true class.
- **Line 18:** Adds the color bar to the map panel with the label `mean T over channels`.
- **Line 20:** Plots a histogram of all gate values in that block.
- **Line 21:** Labels the histogram axes.
- **Lines 22-23:** Lays out and displays the figure.

Cell 24: Class-specific routing heading

Role of this cell. Explains the paper's Figure 3 idea for its 50-layer networks: compare each class's mean gate activity with the overall average. Red cells mean a class transforms more than usual in that layer; blue cells mean it carries more than usual. **Why this matters.** It tests whether the gates are input-dependent or just a fixed layer-wise pattern.

Cell 25: Class-routing heatmap

What this cell does. Subtracts the overall per-layer gate activity from each class's per-layer activity and displays the result as a diverging heatmap.

```
diff = (class_activity - overall_activity).numpy()
limit = max(abs(diff).max(), 0.01)

plt.figure(figsize=(9, 4))
plt.imshow(diff, aspect="auto", cmap="coolwarm", vmin=-limit, vmax=limit)
plt.colorbar(label="class mean T - overall mean T")
plt.yticks(range(len(classes)), classes)
```

```
 8 | plt.xlabel("layer")
 9 | plt.title("Which layers each class uses more or less than average")
10 | plt.tight_layout()
11 | plt.show()
```

Line by line:

- **Line 1:** `diff = (class_activity - overall_activity).numpy()` computes the difference. `class_activity` has shape `(10, 19)` and `overall_activity` has shape `(19,)`, so broadcasting subtracts the overall value from each class.
- **Line 2:** `limit = max(abs(diff).max(), 0.01)` finds the largest absolute difference and uses at least 0.01 so the color scale never has zero width.
- **Line 4:** Creates the figure.
- **Line 5:** `imshow` displays the difference. `coolwarm` is a diverging colormap, and symmetric limits keep zero at the center color.
- **Line 6:** Adds a color bar labeled with the difference in mean gate activity.
- **Line 7:** Puts the 10 class names on the y-axis.
- **Line 8:** Labels the x-axis as layer.
- **Line 9:** Adds the title.
- **Lines 10-11:** Lays out and displays the figure.

Cell 26: Information highway heading

Role of this cell. Explains the cosine-similarity plot. A highway layer can pass its input through almost unchanged. Values close to 1 mean the block copied its input; values near 0 mean it changed the representation substantially.

Cell 27: Layer-to-layer input/output similarity

What this cell does. Runs the fixed probe batch through the model manually, computes the cosine similarity between each block's output and its input, and plots the average similarity per block.

```
 1 | @torch.no_grad()
 2 | def layer_similarity(model, x):
 3 |     model.eval()
 4 |     h = model.stem(x)
 5 |     sims = []
 6 |     for i, block in enumerate(model.blocks):
 7 |         if i == model.break1:
 8 |             h = model.down1(h)
 9 |         if i == model.break2:
10 |             h = model.down2(h)
11 |         h_new = block(h)
12 |         sims.append(F.cosine_similarity(h_new, h, dim=1).mean().item())
13 |         h = h_new
14 |     return sims
15 |
16 |
17 | plt.figure(figsize=(8, 3.5))
18 | plt.plot(range(1, len(highway.blocks) + 1), layer_similarity(highway, probe_x), "o-")
19 | plt.axhline(0, color="gray", linewidth=1)
20 | plt.ylim(-0.2, 1)
21 | plt.xlabel("layer")
22 | plt.ylabel("cosine similarity(output, input)")
23 | plt.title("How much each layer changes its input")
24 | plt.grid(alpha=0.3)
25 | plt.tight_layout()
26 | plt.show()
```

Line by line:

- **Line 1:** `@torch.no_grad()` disables gradients.
- **Line 2:** `def layer_similarity(model, x)` defines the helper.
- **Line 3:** Switches to evaluation mode.
- **Line 4:** Runs the stem to get the first block input.
- **Line 5:** `sims = []` stores one similarity per block.
- **Line 6:** Loops over blocks by index.
- **Lines 7-8:** Before block 6, apply `down1`, matching the model's forward pass.
- **Lines 9-10:** Before block 12, apply `down2`.
- **Line 11:** Applies the block.
- **Line 12:** `F.cosine_similarity(h_new, h, dim=1)` computes similarity along the channel dimension, giving a map per spatial position. `.mean().item()` averages over batch and space and stores a Python float.
- **Line 13:** Updates the input for the next block.
- **Line 14:** Returns one similarity per block.
- **Line 17:** Creates the figure.
- **Line 18:** Plots highway similarity per layer with circle markers.
- **Line 19:** Draws a zero reference line.
- **Line 20:** Fixes the y-range to −0.2 to 1.

- **Line 21:** Labels the x-axis as layer.
- **Line 22:** Labels the y-axis as cosine similarity between output and input.
- **Line 23:** Adds the title.
- **Line 24:** Adds a grid.
- **Lines 25-26:** Lays out and displays the figure.

Cell 28: Lesioning heading

Role of this cell. Explains the paper's Figure 4 lesioning experiment for its 50-layer networks. Close all transform gates of one layer at a time, forcing that layer to copy its input, and measure the test-accuracy drop. A drop near zero means the layer is barely used; a large drop means it is important. **Caveat.** With 10,000 test images, differences below about 0.5 percentage points are close to noise, so small bars should not be over-interpreted.

Cell 29: Layer-lesion experiment

What this cell does. Measures baseline test accuracy, closes each highway block's gates one at a time, records the accuracy drop, restores the block, and plots the drops.

```
baseline_acc = accuracy(highway, test_loader)
drops = []
for block in highway.blocks:
    assert isinstance(block, Highway)
    block.gate_off = True
    drops.append(100 * (baseline_acc - accuracy(highway, test_loader)))
    block.gate_off = False

plt.figure(figsize=(8, 3.5))
plt.bar(range(1, len(highway.blocks) + 1), drops, color="tab:red")
plt.axhline(0, color="gray", linewidth=1)
plt.xlabel("layer with closed gates")
plt.ylabel("accuracy drop (percentage points)")
plt.title("Contribution of each highway layer")
plt.grid(alpha=0.3, axis="y")
plt.tight_layout()
plt.show()
```

Line by line:

- **Line 1:** `baseline_acc = accuracy(highway, test_loader)` measures accuracy with every gate active.
- **Line 2:** `drops = []` will store one drop per layer.
- **Line 3:** Loops over all highway blocks.
- **Line 4:** `assert isinstance(block, Highway)` documents and narrows the type.
- **Line 5:** `block.gate_off = True` forces that block's gate to zero, so the block returns its input.
- **Line 6:** Measures accuracy with the block lesioned and stores the drop in percentage points. `100 * (baseline - lesioned)` is positive when accuracy falls.
- **Line 7:** Restores the block before moving to the next one.
- **Line 9:** Creates the figure.
- **Line 10:** Bar chart of accuracy drop per lesioned layer.
- **Line 11:** Draws a zero reference line.
- **Line 12:** Labels the x-axis as the layer whose gates were closed.
- **Line 13:** Labels the y-axis as accuracy drop in percentage points.
- **Line 14:** Adds the title.
- **Line 15:** Adds a horizontal grid.
- **Lines 16-17:** Lays out and displays the figure.

Cell 30: Feature space heading

Role of this cell. Explains the t-SNE plot. The global average pooled feature before the classifier is the network's learned representation. t-SNE compresses that 128-dimensional space to two dimensions. Well-separated clusters mean the representation is easier to classify. **Caveat.** t-SNE is qualitative. Distances between clusters are not meaningful, and the plot can change with seed and perplexity.

Cell 31: t-SNE of pooled features

What this cell does. Collects the 128-dimensional pooled feature for 2,000 test images, runs t-SNE, and scatters the 2D embedding colored by class.

```
@torch.no_grad()
def penultimate_features(model, loader, limit=2000):
    model.eval()
    feats, labels = [], []
    seen = 0
    for x, y in loader:
        h = model.stem(x.to(device))
```

```
        for i, block in enumerate(model.blocks):
            if i == model.break1:
                h = model.down1(h)
            if i == model.break2:
                h = model.down2(h)
            h = block(h)
        h = h.mean(dim=(2, 3))  # global average pooling
        feats.append(h.cpu())
        labels.append(y)
        seen += y.numel()
        if seen >= limit:
            break
    return torch.cat(feats)[:limit].numpy(), torch.cat(labels)[:limit].numpy()


def tsne_2d(feats):
    return TSNE(n_components=2, perplexity=30, init="pca", random_state=SEED).fit_transform(feats)


feats_highway, labels_highway = penultimate_features(highway, test_loader)
emb_highway = tsne_2d(feats_highway)

fig, ax = plt.subplots(figsize=(8, 5))
for c, name in enumerate(classes):
    mask = labels_highway == c
    ax.scatter(emb_highway[mask, 0], emb_highway[mask, 1], s=5, alpha=0.7, label=name)
ax.set(xticks=[], yticks=[])

handles, names = ax.get_legend_handles_labels()
fig.legend(handles, names, loc="lower center", ncol=5, fontsize=7, markerscale=2)
plt.suptitle("t-SNE of pooled features (2,000 test images)")
plt.tight_layout(rect=(0, 0.08, 1, 1))
plt.show()
```

Line by line:

- **Line 1:** `@torch.no_grad()` disables gradients.
- **Line 2:** `def penultimate_features(model, loader, limit=2000)` defines the feature collector.
- **Line 3:** Switches to evaluation mode.
- **Line 4:** Initializes feature and label lists.
- **Line 5:** `seen = 0` counts how many samples have been collected.
- **Line 6:** Loops over the test loader.
- **Line 7:** Runs the stem.
- **Line 8:** Loops over blocks by index.
- **Lines 9-10:** Before block 6, apply `down1`.
- **Lines 11-12:** Before block 12, apply `down2`.
- **Line 13:** Applies the block.
- **Line 14:** `h = h.mean(dim=(2, 3))` applies global average pooling, giving `(batch, 128)`.
- **Line 15:** Stores the features on CPU.
- **Line 16:** Stores the labels.
- **Line 17:** Adds the batch size to `seen`.
- **Lines 18-19:** Stops once at least 2,000 samples are collected.
- **Line 20:** Concatenates batches, trims to exactly 2,000, and converts to NumPy for scikit-learn.
- **Line 23:** `def tsne_2d(feats)` defines the t-SNE wrapper.
- **Line 24:** Runs t-SNE with two components, perplexity 30, PCA initialization, and the global seed. Perplexity controls the effective neighborhood size; 30 is standard for a few thousand points.
- **Line 27:** Collects highway features from the test set.
- **Line 28:** Computes the 2D embedding.
- **Line 30:** Creates one figure.
- **Line 31:** Loops over the 10 classes.
- **Line 32:** `mask = labels_highway == c` selects the points of class `c`.
- **Line 33:** Scatters those points with small, semi-transparent markers.
- **Line 34:** Removes axis ticks because t-SNE coordinates have no meaning.
- **Line 36:** Collects legend handles and class names from the axis.
- **Line 37:** Places a shared legend below the plot.
- **Line 38:** Adds the title.
- **Line 39:** Leaves space at the bottom for the legend.
- **Line 40:** Displays the figure.

Cell 32: Predictions heading

Role of this cell. A heading for the prediction grid. It is a qualitative sanity check, not a quantitative evaluation.

Cell 33: Prediction grid

What this cell does. Takes one test batch, keeps eight images, runs the highway model, and shows each image with its true label, predicted label, and confidence.

```
sample_iter = iter(test_loader)  # keep the iterator alive for a clean worker shutdown
images, labels = next(sample_iter)
images, labels = images[:8], labels[:8]  # one batch holds 512, show 8
with torch.no_grad():
    probs = highway(images.to(device)).softmax(dim=1).cpu()
labels = labels.tolist()
preds = probs.argmax(dim=1).tolist()
confs = probs.max(dim=1).values.tolist()

fig, axes = plt.subplots(2, 4, figsize=(10, 5.5))
for ax, img, label, pred, conf in zip(axes.flat, images, labels, preds, confs, strict=True):
    rgb = (img * torch.tensor(STD)[:, None, None] + torch.tensor(MEAN)[:, None, None]).clamp(0, 1)
    ax.imshow(rgb.permute(1, 2, 0))
    ax.set_title(f"true: {classes[label]}\npred: {classes[pred]} ({conf:.0%})", fontsize=9)
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Line by line:

- **Line 1:** Creates a test iterator and keeps it in `sample_iter` so its workers shut down cleanly.
- **Line 2:** Gets the first test batch. The test loader does not shuffle, so this is always the same batch.
- **Line 3:** Keeps only the first eight images and labels.
- **Lines 4-5:** Runs the model without gradients, applies softmax to turn logits into probabilities, and moves the result to CPU.
- **Line 6:** Converts labels to Python integers.
- **Line 7:** `probs.argmax(dim=1).tolist()` gives the predicted class indices.
- **Line 8:** `probs.max(dim=1).values.tolist()` gives the confidence of each prediction.
- **Line 10:** Creates a 2×4 grid of axes.
- **Line 11:** Iterates over images, labels, predictions, and confidences. `strict=True` checks that all sequences have the same length.
- **Line 12:** Reverses normalization with `img * STD + MEAN`, broadcasting per channel, and clamps to 0 to 1 for display.
- **Line 13:** Converts from channels-first to channels-last for matplotlib and displays the image.
- **Line 14:** Adds a title with the true class, predicted class, and confidence as a percentage.
- **Line 15:** Hides the axes.
- **Lines 17-18:** Lays out and displays the grid.

Reference tables and likely questions

Tensor shape flow

Stage	Tensor shape	What happens
Input	`(B, 3, 32, 32)`	CIFAR-10 batch.
Stem	`(B, 32, 32, 32)`	Plain 3×3 convolution, 3 to 32 channels, ReLU.
Stage 1	`(B, 32, 32, 32)`	Six highway blocks at 32 channels.
Down 1	`(B, 64, 16, 16)`	Plain 3×3 convolution, stride 2, 32 to 64 channels, ReLU.
Stage 2	`(B, 64, 16, 16)`	Six highway blocks at 64 channels.
Down 2	`(B, 128, 8, 8)`	Plain 3×3 convolution, stride 2, 64 to 128 channels, ReLU.
Stage 3	`(B, 128, 8, 8)`	Seven highway blocks at 128 channels.
Pool	`(B, 128)`	Global average pooling over the 8×8 spatial grid.
Head	`(B, 10)`	Linear classifier producing 10 logits.

Gate tensor shape

Inside a highway block, `T` has shape `(B, C, H, W)`. There is one gate value for every image, channel, and spatial position. The analysis reduces it in three ways: `mean(dim=(1,2,3))` gives one number per image; `mean()` gives one number per layer; `mean(dim=0)` on a single image gives a spatial map averaged over channels.

Parameter breakdown

Part	Parameters	Why
Stem 3 to 32	896	3×3×3×32 weights plus 32 biases.

Part	Parameters	Why
6 blocks at 32	110,976	Each block has two convs: 2 × (32×32×3×3 + 32) = 18,496.
Down 1, 32 to 64	18,496	32×64×3×3 + 64.
6 blocks at 64	443,136	Each block: 2 × (64×64×3×3 + 64) = 73,856.
Down 2, 64 to 128	73,856	64×128×3×3 + 128.
7 blocks at 128	2,066,176	Each block: 2 × (128×128×3×3 + 128) = 295,168.
Head 128 to 10	1,290	128×10 weights plus 10 biases.
Total	**2,714,826**	Printed by the notebook.

Hyperparameter rationale

Setting	Value	Why this value
Seed	`0`	Reproducible initialization and data order.
Batch size	`128`	Paper reproduction and standard SGD batch size.
Epochs	`25`	A short pedagogical run. The authors' released Caffe config uses 200k iterations, about 400 epochs at batch 100.
Initial LR	`0.1`	High CIFAR-10 rate; the authors' released config uses 0.025 with Nesterov momentum.
LR decay	`0.89` per epoch	Short-run exponential schedule; the paper's config uses step decay at 100k/150k/175k iterations.
Momentum	`0.9`	Standard SGD momentum; the paper uses SGD with momentum.
Weight decay	`5e-4`	Practical CIFAR-10 regularization; not stated explicitly in the paper.
Gradient clipping	`10`	Stability addition; not in the paper.
Gate bias	`-2`	Paper's negative initialization. `sigmoid(-2)` ≈ `0.12`.
Blocks	`19`, split `6/6/7`	Depth-20 convention with one stem.
Channels	`32/64/128`	Compact widths for an 8 GB GPU.
Workers	`2`	Parallel data loading; use 0 on Windows if needed.

Depth counting

There are three different counts, and they are all correct in different contexts:

- **Notebook depth:** 20 hidden layers. One plain stem plus 19 highway blocks. This is the depth-20 hidden-stack convention, not the paper's 19-layer CIFAR-10 architecture (14 highway layers plus plain convolutions).
- **Highway blocks:** 19. Each block contains a transform convolution `H` and a gate convolution `T`.
- **Physical convolution layers:** 41. One stem convolution, 38 block convolutions, and two downsampling convolutions. The classifier is a linear layer, not a convolution.

Likely questions and short answers

Why use convolution instead of a fully connected layer?

The paper's CIFAR-10 experiments use convolutional highway layers. Convolutions share weights across spatial positions, preserve local structure, and need far fewer parameters than a dense layer on 32×32×3 inputs.

Why is the kernel 3×3 with padding 1?

A 3×3 convolution with padding 1 preserves the height and width. The highway equation adds the input to the transformed output, so both must have the same spatial size. Padding 1 makes that true.

Why can a highway block not change channels?

The carry term is `x * (1 - T)`. If `H(x)` had a different number of channels from `x`, the addition would be invalid. Channel-changing layers are therefore plain convolutions, as in Section 2.1 of the paper.

Why 19 blocks?

The notebook's depth-20 hidden stack is one plain layer followed by 19 highway layers. The paper's released 19-layer CIFAR-10 network is different: it has 14 highway layers interleaved with plain convolutions. Nineteen blocks give a deeper gated stack; the run takes about 13 to 15 minutes.

Why are there 41 convolutions if the depth is 20?

Each highway block has two convolutions, `H` and `T`. The paper counts a highway block as one layer. Nineteen blocks therefore give 38 convolutions, plus one stem and two downsampling convolutions, for 41 physical convolutions.

Why is the transform gate a sigmoid?

The gate must be between 0 and 1 so the layer is a convex combination of the transformed input and the original input. A sigmoid guarantees that range and is smooth and differentiable.

Why start the gate bias at −2?

`sigmoid(-2) ≈ 0.12`, so gates start mostly closed. The deep network initially behaves like a shallower, carry-heavy network and adapts the gates during training. This follows the paper's Section 2.2 recommendation of a negative initial bias.

Why He initialization?

The transform branch uses ReLU. He normal initialization is designed to preserve signal variance in ReLU networks. The paper initializes the non-gate weights with the He et al. scheme.

Why reset the gate bias after initialization?

`init_weights` zeros every bias, including the gate biases. The second loop restores the −2 value for every highway block. Without that reset, the negative initialization would be lost.

Why use SGD with momentum instead of Adam?

The paper trains with SGD and momentum. Adam would be a different optimizer and would make the comparison to the paper less direct.

Why learning rate 0.1?

It is a high CIFAR-10 rate that stays stable here because the highway shortcuts preserve gradient flow. The authors' released config uses base LR 0.025 with Nesterov momentum and step decay.

Why decay 0.89 instead of 0.98?

The paper's released config uses step decay, not a per-epoch exponential. For this 25-epoch run, 0.89 is a short-run choice that brings the learning rate down to about 0.005.

Why 25 epochs?

It keeps the run near 13 to 15 minutes on the RTX 3060 Ti. The tradeoff is lower accuracy and less gate convergence than the paper's 400-epoch run.

Why weight decay 5e-4?

It is a practical CIFAR-10 regularization value that helps convolutional models generalize. The paper does not state it explicitly, so it is a documented addition.

Why gradient clipping at 10?

Deep ReLU networks with SGD can occasionally produce very large gradients. Clipping the total norm at 10 keeps training stable. The paper does not mention clipping, so it is an implementation addition.

Why is there no softmax before the loss?

`CrossEntropyLoss` expects raw logits and applies log-softmax internally. This is numerically more stable than applying softmax first.

Why global average pooling?

It turns the final `(B, 128, 8, 8)` feature map into `(B, 128)` without extra parameters. It also makes the classifier independent of the exact spatial position of features.

Why is `shuffle=True` only for training?

Shuffling prevents the optimizer from seeing a fixed batch order and improves SGD. Test order is kept fixed so results and probe batches are reproducible.

Why `pin_memory` only on CUDA?

Pinned memory speeds up host-to-GPU transfers. It has no benefit on CPU and can produce a warning there.

Why `set_to_none=True` in `zero_grad`?

It sets gradients to `None` instead of allocating zero tensors. This is slightly cheaper and is the recommended PyTorch default for many training loops.

Why `detach` when storing gates?

The stored gate tensor is for inspection only. Detaching it removes it from the autograd graph, so it does not keep graph memory alive across iterations.

Why `retain_grad` in the gradient experiment?

The gradient experiment needs the gradient of non-leaf tensors, the inputs to each block. PyTorch does not keep those by default, so `retain_grad()` is called on each one.

Why does the gradient helper repeat the downsampling logic?

The helper runs the model manually to capture intermediate tensors. It must apply `down1` and `down2` at the same block indices as `Net.forward`, or the channel counts will not match.

Why average the gate map over channels?

A 64-channel 16×16 gate tensor is too dense to read as one image. Averaging over channels gives a readable spatial map. The histogram still shows the full distribution.

Why does lesioning use accuracy instead of loss?

Accuracy is easy to interpret. The downside is noise: with 10,000 test images, differences below about 0.5 percentage points are near the noise floor. Training or test loss would be more sensitive.

Why is t-SNE qualitative?

t-SNE preserves local neighborhoods, not global distances. Cluster sizes and between-cluster distances are not meaningful, and the result changes with seed and perplexity.

Why is there only one model now?

The plain baseline was removed by request. The notebook now studies the highway model alone. It can no longer directly demonstrate that highway networks beat plain networks at depth.

Known inconsistencies and caveats

- **No direct depth comparison.** Without a plain baseline, the notebook cannot prove the paper's depth claim. It can only show that the highway model trains and how its gates behave.
- **Short training.** Twenty-five epochs is much shorter than the paper's released config, which uses about 400 epochs. Gates may not fully converge, and gate/lesioning plots may be noisy.
- **Compact architecture.** The model is not an exact FitNet reproduction. It uses 19 highway blocks in a depth-20 hidden stack, while the paper's 19-layer CIFAR-10 net has 14 highway layers plus plain convolutions.
- **Gate-bias direction.** In this run the learned biases move toward zero, so the gates open on average. The paper's much deeper networks often became more negative and selective. The difference is expected given the architecture and the short run, but it means the gate plots are not a direct reproduction of Figure 2.
- **Lesioning noise.** Small accuracy drops should not be over-interpreted.
- **Single-image gate map.** Averaging over channels can hide per-channel selectivity. The histogram is the more reliable part.

Paper map

Paper item	Where it appears in the notebook
Equation (3), the highway layer	Cell 0 theory, cell 7 `Highway.forward`, line 17.
Equations (4) and (5), identity and Jacobian cases	Cell 0 derivation, `gate_off` lesioning in line 14-15.
Section 2.1, convolutional highways and plain dimensionality changes	Cell 8 description and cell 9 plain stem and downsampling convolutions.
Section 2.2, negative gate bias	Cell 3 `GATE_BIAS = -2` and cell 7 line 7.
Section 3.1, MLP depth study	Not reproduced. The notebook uses a CIFAR-10 convolutional model.
Table 2, CIFAR-10 convolutional highways	19 highway blocks plus a plain stem, 2.71M parameters. This is a compact variant, not an exact FitNet layout.
Figure 2, gate biases and activity	Cells 18-23.
Figure 3, class-specific routing	Cells 24-25.
Figure 4, layer lesioning	Cells 28-29.
Longer paper 1507.06228	Source for the convolutional highway construction and the gate/lesioning analyses.

One-paragraph summary to memorize

The notebook builds a depth-20 convolutional highway network for CIFAR-10. A plain stem changes the input to 32 channels. Three stages of highway blocks use 32, 64, and 128 channels, with plain strided convolutions between stages. Each highway block computes `y = H(x) * T(x) + x * (1 - T(x))`, where `H` and `T` are 3×3 convolutions with padding 1 and the gate bias starts at −2. The model has 19 highway blocks, 41 physical convolutions, and 2,714,826 parameters. It is trained with SGD momentum 0.9, learning

rate 0.1, weight decay 5e-4, decay 0.89 per epoch, gradient clipping 10, batch size 128, for 25 epochs. The analysis plots gradient flow, gate biases and activity, gate evolution, single-image gate maps, class routing, input-output similarity, lesioning, and t-SNE features.